

ORIGINAL ARTICLE

Learning Bayesian Network Structures Without Variable Ordering Influence: A Markov Blanket-Based Approach

Lívia Carvalho Dâmaso¹ | Edimilson Batista dos Santos¹ | Fabio Gagliardi Cozman²

¹Department of Computer Science (DCOMP), Federal University of São João del-Rei, Minas Gerais, Brazil | ²Escola Politécnica, University of São Paulo, São Paulo, Brazil

Correspondence: Edimilson Batista dos Santos (edimilson.santos@ufsj.edu.br)

Received: 24 March 2025 | **Revised:** 21 October 2025 | **Accepted:** 12 November 2025

Keywords: Bayesian networks | Markov blanket | structure learning

ABSTRACT

Several methods in the literature address the discovery, from data, of a directed acyclic graph that is the structure of a Bayesian network. This is a challenging task, due to the combinatorial nature of the space of graphs, which grows exponentially with the number of variables. This paper proposes a new method to learn Bayesian network structures, based on Markov blankets. The proposed method, called DMBBN (Dynamic Markov Blanket Bayesian Network), builds a graph from a set of local structures that are induced based on the Markov blanket of each variable of interest. The local structures are combined to generate a single Bayesian network structure, without repetition of nodes and without cycles. The experiments carried out show that DMBBN is promising, especially in large datasets, as it does not depend on a prior ordering of the variables.

1 | Introduction

A key challenge in Artificial Intelligence is to build systems that act rationally. Black-box solutions restricted to predictive optimization are not always adequate to inform decision-making in domains that require transparency and tractability, such as government policy and healthcare [1]. Pearl and Mackenzie [2] argue that rational agents should be able to reason about causation to go beyond mere prediction. In addition, interest in causality has increased in the machine learning community in recent years, as causal modeling can lead to more robust models.

Bayesian Networks provide an important tool for reasoning under uncertainty, and they offer tools to handle many complex causal systems [3, 4]. They have been used in many different domains, for example, in protein [5] and gene [6] networks in biology, psychosis [7], cancer treatment [8] in healthcare, engineering fault diagnosis [9] and air pollution modeling [10].

Bayesian networks are well-suited to model complex non-deterministic systems with causal connections, given that some relevant assumptions are adopted.

Bayesian networks combine graph theory and probability theory, as a Bayesian network consists of two parts: The graphical structure (a Directed Acyclic Graph [DAG]) and the numerical parameters (conditional probability tables). The DAG in a Bayesian network conveys the direct dependence relationships between variables or, in a Bayesian network with a causal interpretation, direct cause-and-effect relationships. Such relationships play an important role in how we perceive and make sense of the world around us, how we act upon it, and ultimately, how we understand ourselves [11]. A Bayesian network is also specified through probabilities that capture the strength of the relationships among variables. Determining these probabilities is usually much easier than recovering the DAG accurately. When the DAG is unknown, a structure learning problem must be solved [12].

However, learning or inducing a DAG is a complex problem, considered NP-hard [13, 14]. Thus, many researchers have sought to develop new methods that can optimize structure learning, including with the aid of other techniques, as presented by Kitson et al. [1].

In this paper, the goal is to explore the concept of the Markov blanket to propose an algorithm capable of learning Bayesian network structures. The proposed method is called DMBBN (Dynamic Markov Blanket Bayesian Network); it builds the DAG from a set of local structures that are induced from the Markov blanket of each variable in the problem. An advantage of the proposed method is that it is not influenced by a previous ordering of variables.

The proposed algorithm has been tested on eleven datasets, ranging from small to large, with promising results.

The remainder of the paper is organized as follows. Section 2 presents the fundamentals of Bayesian networks and structure learning. Section 3 brings related work. Section 4 presents the proposed DMBBN algorithm. Section 5 discusses the results obtained in the experiments. Finally, Section 6 concludes our paper and points out future work.

2 | Bayesian Networks

Bayesian networks provide a compact representation of a joint probability distribution, making it possible to apply probability theory effectively to systems involving thousands of variables [1].

A Bayesian network consists of two important components:

- A Directed Acyclic Graph (DAG) $G = (V, E)$ —the graphical structure—where $V = \{X_1, X_2, \dots, X_n\}$ is a set of nodes, and E is a set of ordered pairs of distinct elements of V , representing the dependency relations between the n variables. Each node is associated with a random variable, also referred to as an attribute; these words are not distinguished in this paper. The elements of E are called edges (or arcs). If there is a directed edge from node X_i to node X_j , then X_i is a parent of X_j .
- A set of Conditional Probability Tables (CPT)—or numerical parameters—that carries the probabilistic effect that the set of parents of X_i has on X_i .

In a Bayesian network, the absence of some edges in the DAG implies the existence of conditional independence relationships between the variables, while their presence suggests the existence of direct dependence relationships. The probability tables, known as the quantitative components of the Bayesian network, represent conditional probability measures carrying the strength of such dependencies.

Figure 1 depicts the structure of a Bayesian network and shows the direct relationship between its variables, connected by directed edges—variable X_i is directly influenced by X_j if there is an edge from X_i to X_j . One may attribute the relationship between variables to a causal origin [4]; a variable may be

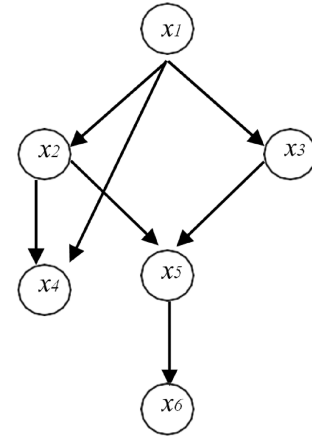


FIGURE 1 | Example of Bayesian network structure.

a cause to one or more variables, and a variable may be causally influenced by one or more variables.

The joint distribution for all variables in a Bayesian network is then a product of conditional distributions. In general, we have $P(x_1, \dots, x_m) = \prod_{i=1}^m P(x_i | \pi_{x_i})$, where m is the number of variables, $P(x_1, x_2, \dots, x_6)$ is the joint probability of $X = \{x_1, x_2, \dots, x_6\}$ and X is a set of random variables. $P(x_i | \pi_{x_i})$ is the conditional probability of x_i , given that x_j is known, and π_{x_i} is the set of parents of x_i . This compact expression is a key advantage of Bayesian networks when compared to other formalisms that represent uncertainty. For the network in Figure 1, we have $P(x_1, x_2, \dots, x_6) = P(x_6 | x_5)P(x_5 | x_2, x_3)P(x_4 | x_1, x_2)P(x_3 | x_1)P(x_2 | x_1)P(x_1)$.

We can define a set of variables in the Bayesian network that surely isolates a given variable from all other variables, as follows:

Definition 1. Suppose V is a set of random variables, P is their joint probability distribution, and $X \in V$. Then a Markov blanket of X , $MB(X)$, is a set of variables such that X is conditionally independent of all other variables given $MB(X)$.

We then have [15].

Theorem 1. Suppose a Bayesian network G satisfies the Markov condition given in Definition 1. For each variable X in G , the set of all parents of X , children of X , and parents of children of X is a Markov blanket of X ($MB(X)$).

The Markov blanket of a variable X can be used to select a subset of relevant variables that actually carry information about X [16]. This concept plays an important role in our proposed method.

2.1 | Learning Bayesian Network Structures

The main challenge when learning a Bayesian network is to build the graph (structure) accurately. Learning the structure of a Bayesian network from data is an NP-hard problem [13, 14], with a solution space that grows exponentially with the number of variables. Learning the numerical parameters from data is usually taken to be a simpler task.

Broadly, there are two classes of algorithms that can be used to learn structures from data. The first class consists of algorithms based on conditional independence tests. An example is the PC [17] algorithm, which uses statistical tests of marginal and conditional independence to identify dependencies and interdependencies in the data and obtain possible causal relationships. The second class of algorithms consists of score-based ones, such as the K2 [18], which perform a graph search over a set of possible graphs and have a scoring metric that indicates how well each visited graph fits the data. Hybrid algorithms, which combine elements of both approaches, are often considered a third major class of structure learning. Note that the structure can also be specified by human experts in a domain of interest; we do not contemplate this possibility in this paper.

2.1.1 | The K2 Algorithm

The K2 algorithm [18] is a heuristic search method designed to learn the structure of Bayesian networks. It receives, as input, a set of data, an ordering of variables, and an upper bound on the number of parents a node may have, and generates, as output, the structure of the network. The K2 algorithm (see Algorithm 1) relies on the variable ordering to define the search space. The algorithm employs a function $Pred(X_i)$ that returns the set of nodes that precede x_i in the node ordering. The ordered list (containing all variables) states that only variables positioned before a given variable X_i can be parents of X_i . Thus, the first variable in the list has no parents; that is, it is a root node in the Bayesian network.

The algorithm uses greedy steps to find the best structure. It starts by assuming that nodes have no parents. Then, starting from the second variable in the ordered list (the first is a root node), the

possible parents are tested, and those that maximize the probability of the structure are added to the network. This process is repeated for all variables until the best possible structure is found.

The metric (score) applied by K2 to test each set of possible parents, for each variable, is given by:

$$g(i, \pi_i) = \prod_{j=1}^{q_i} \frac{(r_i - 1)!}{(N_{ij} + r_i - 1)!} \prod_{k=1}^{r_i} N_{ijk}!, \quad (1)$$

where each variable X_i ($i = 1, \dots, n$) has r_i possible values ($v_{i1}, v_{i2}, \dots, v_{ir_i}$), and a set of parents π_i . Also, q_i is the number of instantiations of π_i ; N_{ijk} is the number of objects in the dataset D , where X_i has the value v_{ik} and π_i is instantiated as w_{ij} , which represents the j th instantiation relative to D of π_i . Finally, $N_{ij} = \sum N_{ijk}$.

2.1.2 | The DMBC Algorithm

The Dynamic Markov Blanket Classifier (DMBC) [19] was derived from the K2 algorithm to learn Bayesian networks aimed at classification problems. The DMBC algorithm, as shown in Algorithm 2, is basically a version of the K2 algorithm designed to exploit the Markov blanket of the class variable (CMB), so that structure induction is faster and more accurate when it comes to estimates of class probabilities.

The main idea of DMBC is to exclude, from the possible structures (DAG) of the Bayesian network to be built, those that have variables that do not belong to the dynamically created CMB. DMBC (Algorithm 2) differs from K2 (Algorithm 1) in lines 1, 7, 10, 11, 15, and 16. These lines are extra constraints that lead to a reduction in the number of possible structures to be tested. Although DMBC is a heuristic method and therefore cannot guarantee the discovery of the true underlying causal structure, studies by Santos et al. [19] demonstrate its ability to yield a highly effective and practical approximation.

Using only variables within the Markov blanket of a class variable can significantly reduce the computational time needed to learn a Bayesian network. Furthermore, as DMBC tends to reduce the number of variables in the learned network, DMBC can be seen as an alternative when one does not want to spend significant time searching for the best global structure.

3 | Related Work

Kitson et al. [1] present a review of Bayesian network structure learning algorithms, which aim to learn the graph from data. The authors review seventy-four algorithms, encompassing prototypical, well-established, and state-of-the-art approaches, where the focus is to search or constrain the finite discrete space of possible graphs in some way. The review provides a consistent description of the basic approach of each algorithm, highlighting key similarities and differences. Based on their comprehensive study, the authors conclude that no single structure learning algorithm is universally superior and note the inconsistency within the existing literature regarding which methods are the most accurate.

ALGORITHM 1 | K2 algorithm.

Require: Set of n nodes, a node ordering, an upper bound u on the number of parents a node may have, and a dataset D containing m cases.

Ensure: For each node, a printout of the parents of the node.

```

1: for  $i := 1$  To  $n$  do
2:    $\pi_i := \{\}$ 
3:    $P_{old} := g(i, \pi_i)$  (see Eq. 1)
4:   OKToProceed := true
5:   while OKToProceed and  $\pi_i < u$  do
6:     let  $z$  be the node  $Pred(X_i) - \pi_i$  that
       maximizes  $g(i, \pi_i \cup \{z\})$ 
7:      $P_{new} = g(i, \pi_i \cup \{z\})$ 
8:     if  $P_{new} > P_{old}$  then
9:        $P_{old} := P_{new}$ 
10:       $\pi_i := \pi_i \cup \{z\}$ 
11:    else
12:      OKToProceed := false
13:    end if
14:  end while
15:  write('Node:',  $X_i$ , 'Parents of this
    node:',  $\pi_i$ )
16: end for

```

ALGORITHM 2 | DMBC algorithm.

Require: Set of n nodes, a node ordering (having the class as the first node), an upper bound u on the number of parents a node may have, and a dataset D containing m cases.

Ensure: For each node, a printout of the parents of the node.

```

1:  $CMB := \{\}$  ▷ (Class' Markov blanket)
2: for  $i := 1$  To  $n$  do
3:    $\pi_i := \{\}$ 
4:    $P_{old} := g(i, \pi_i)$  (see Eq. 1)
5:    $OKToProceed := \text{true}$ 
6:   while  $OKToProceed$  and  $\pi_i < u$  do
7:     let  $z$  be the node in  $\text{Pred}(X_i) - \{\pi_i \cup CMB\}$ 
       that maximizes  $g(i, \pi_i \cup \{z\})$ 
8:      $P_{new} = g(i, \pi_i \cup \{z\})$ 
9:     if  $P_{new} > P_{old}$  then
10:      if ( $z$  is parent of any class
        child) or ( $z$  is a class child) then
11:         $CMB := CMB \cup \{z\}$ 
12:         $P_{old} := P_{new}$ 
13:         $\pi_i := \pi_i \cup \{z\}$ 
14:      end if
15:      if ( $\text{not\_class\_child}(i)$  and  $\text{class}$ 
         $\notin \pi_i$ ) then
16:        break
17:      end if
18:    else
19:       $OKToProceed := \text{false}$ 
20:    end if
21:  end while
22:  write('Node:',  $X_i$ , 'Parents of this
    node:',  $\pi_i$ )
23: end for

```

In another study, Scutari et al. [20] evaluate different classes of Bayesian network structure learning algorithms. From the experiments, the authors found that constraint-based algorithms are less accurate than tabu search (but not simulated annealing) for both small and large sample sizes; however are more accurate than other score-based algorithms in many simulation scenarios. The results have shown no difference in accuracy between constraint-based and hybrid algorithms. The authors also point out that tabu search, as a scoring-based algorithm, is often faster than most constraint-based and hybrid algorithms. They also claim that hybrid algorithms are not faster than constraint-based or score-based algorithms; there was no consistent ranking of algorithms in these classes across different simulation settings. Finally, no systematic difference in the ranking of different classes of algorithms in terms of speed and accuracy was observed for any class of algorithms for small networks compared to large networks. These results, which were confirmed in both simulated and complex real-world data, provide guidance to further studies; the authors do not exclude the existence of other sources of confounding, such as tuning parameters, which should be investigated further.

The K2 algorithm is recognized as one of the most frequently used methods for structure learning [21]. However, its overall performance is heavily influenced by the ordering of the nodes

(variables or attributes). This node ordering defines a topological constraint on the structure; this is often taken as the order in which variables appear as columns in the input data table, meaning a variable can only be linked to parents on its left. Discovering an effective variable ordering is crucial for the successful application of the K2 algorithm.

Kitson and Constantine [22] discuss the impact of node ordering on learning Bayesian network structures. The authors suggest that variable ordering has a much stronger effect on structure learning performance than previously assumed, especially when compared with other factors on which algorithms are usually tested for sensitivity.

Several algorithms for node ordering can be found in the literature. They are often divided into two groups: Evolutionary algorithms and heuristic algorithms. With respect to evolutionary algorithms, Larranaga et al. [23], Hsu et al. [24], and Romero et al. [25] provide extensive experimental results through various crossover and mutation techniques. With respect to heuristic methods, Hruschka and Ebecken [26] proposed ordering variables using statistical tests and information gain; Chen et al. [27] incorporated information theory and exhaustive search functions; Ko and Kim [28] introduced a measure to assess conditional frequency for node ordering; Aghdam et al. [29] proposed the SORDER algorithm to select the sequential ordering of nodes; Ai [30] used information entropy to order the nodes; Behjati and Beigy [31] presented an algorithm that builds a graph from data and then extracts strongly connected components that are used to generate an initial ordering of nodes, which is passed to K2; Benmohamed et al. [21] proposed an improvement to the K2 algorithm based on extending the parent search space to the parent and child search space as a main step, and an information theory-centered node ordering method. In short, there are many proposals in the literature.

In their comprehensive review, Kitson et al. [1] also dedicate attention to node ordering algorithms for structure learning. They discuss potential benefits in searching over the ordering space; however, they highlight that, in general, a DAG can be consistent with multiple orderings, just as an ordering can be consistent with multiple DAGs.

Given this context, this paper proposes a new score-based algorithm that is not influenced by variable ordering. The main idea is to exploit Markov blankets. In fact, Kitson et al. [1] already state that Markov blankets provide a causal approach to variable selection and that many local structure learning algorithms have used them for classification problems, such as Santos et al. [19]. Within Bayesian network structure learning, local structures may be learned for each node and then merged to form the entire DAG. Kitson et al. also present some constraint-based algorithms which use the concept of Markov blanket, such as Grow-Shrink (GS), IAMB, and its Inter-IAMB variant, Fast-IAMB. Nevertheless, these algorithms return a set of local structures rather than a whole integrated graph.

Tabar et al. [16] used Markov blankets to find a set of candidate parents for each node as input to the K2 algorithm, thus restricting the search for an ordering. First, they determine a Markov blanket for each variable using L1-regularized estimation, and

then they determine the candidate parents for each variable using a novel scoring function. A set of candidate parents is obtained for each variable; these candidate parents are used as input to the K2 algorithm.

In this paper, we propose DMBBN, a novel method for inferring Bayesian network structures. DMBBN uses the Markov blanket of each variable to construct individual local structures, which are then combined to generate the final structure. DMBBN evaluates this final structure using the K2 scoring function (Equation 1). Unlike many other structure learning approaches, DMBBN does not require a pre-defined ordering of variables, offering a significant practical advantage.

4 | DMBBN: Dynamic Markov Blanket Bayesian Network

The Dynamic Markov Blanket Bayesian Network (DMBBN) algorithm exploits Markov blankets to learn local structures and to build a final global structure. DMBBN is a score-based algorithm, unlike Grow-Shrink (GS), IAMB (and its variant Inter-IAMB) algorithms, which also use the Markov blanket, but are based on conditional independence tests. DMBBN uses the g function (Expression (1)) to induce local structures, which are built with the help of the *Modified-DMBC* algorithm—the latter is inspired by the DMBC algorithm. Another key element of DMBBN is the strategy for searching for the parents of a variable, which does not follow a variable ordering, as in the K2 and DMBC algorithms, or even in the IAMB and Inter-IAMB algorithms. In fact, DMBBN does not require a prior ordering of the variables. When focusing on a variable in the search for parents, DMBBN analyzes all other variables in the domain (and not just those to the left of the chosen variable, such as K2 and DMBC) and selects the u most suitable parents (that are part of the Markov blanket of the variable). This enables a more comprehensive exploration of the search space and the selection of optimal parents for each variable.

The DMBBN algorithm receives as input the dataset (D) and the maximum number of parents (u) for each variable, and returns as output the structure of the Bayesian network (a DAG), as shown in Algorithm 3.

The DMBBN algorithm operates in two main steps:

1. Initially, the DMBBN algorithm induces n local structures (n is the number of variables in the domain). Each local structure is generated by searching the Markov blanket for each variable X_i ($i = 1, \dots, n$) in X , executing the *Modified-DMBC* algorithm.
2. The local structures are combined to generate a single Bayesian network structure, without repetition of nodes (attributes) and without cycles. To do this, the algorithm initially evaluates edges in local structures to solve direction ties and to select the dominant direction; it then applies the adapted Kruskal algorithm to avoid cycles.

Sections 4.1 and 4.2 present each step in more detail.

4.1 | Induction of Local Structures

The *Modified-DMBC* algorithm, shown in Algorithm 4, is responsible for inducing local structures. It is a variant of the DMBC [19] discussed in Section 2.1.2. Here, at each step, the class variable is replaced by the first variable of interest.

The main change in *Modified-DMBC* is the fact that it searches for the parents of a variable in the entire set of variables and not just to the left of it, as in the original DMBC. Thus, all variables are examined in the search for the best parents for the variable of interest, and a prior ordering is not necessary.

To induce the n local structures corresponding to the Markov Blanket of each variable X_i ($i = 1, \dots, n$) in the variable X , the algorithm iterates through each variable individually. For a chosen variable X_i , it is temporarily placed in the first position of the vector X to serve as the root node of the local structure. Initially, all nodes are assumed to have no parents. The algorithm then proceeds through the remaining variables sequentially. Starting with the second variable in the reordered vector, it tests whether the first is its parent. If so, other possible parents are tested, and those that maximize the probability of the structure being consistent with the database are added. This process is repeated for all variables until all local structures, centered around X_i as the initial root, are complete.

The metric (score) applied by the algorithm to test each set of possible parents, for each variable, is the same as that used by the K2 and DMBC algorithms, given by the g function (Expression (1)).

Note that the *Modified-DMBC* algorithm searches for structures that map the relationships between the first variable in the input and the other variables. Therefore, even though it uses Markov Blankets, the algorithm can include variables outside the actual blanket in the learned structures. These local structures are later combined to build the final structure.

Figure 2 illustrates this first step of *Modified-DMBC*, which shows eight local structures for the asia dataset, generated from the Bayesian network available in the *bnlearn*¹ repository.

4.2 | Induction of the Final DAG

To perform step 2 in DMBBN, the following procedure is adopted. First, edge candidates are pre-processed by resolving direction ties (removing edges with equal frequency in both directions) and selecting the dominant direction based on frequency counts from local structures. To be more precise: Take a pair of nodes i and j . Check all existing edges connecting these two nodes in all previously created local structures. The edge ij is initially inserted into the DAG with a weight equal to 1, and the weight increases with each appearance.

For each edge ij , the existence of an edge ji is verified. If there is such an edge, the weight of the two edges (ij and ji) are compared, and the strongest one is kept. If their weights are equal, the two edges are canceled and disregarded in the construction of the final DAG. Only the edges remaining after this check are kept in the final DAG.

ALGORITHM 3 | DMBBN algorithm.

Require: Dataset D containing m cases and an upper bound u on the number of parents a node may have.

Ensure: Bayesian network structure (or final DAG).

```

1: adjMatrix := matrix_adjacency
2:   ▷ For each variable  $X_i$  in the domain, ( $i=1,\dots,n$ ), induce a local structure which
   represents the Markov blanket of  $X_i$ 
3: for  $i := 1$  To  $n$  do
4:   ▷ Build local DAG for each  $x_i$ , running Modified-DMBC.
5:   Run Modified-DMBC   ▷  $x_i$  must be the first variable in the list on each call.
6:   for each edge  $ij$  in the local DAG do
7:     Increment the value at position  $[i][j]$  of the adjMatrix by 1
8:   end for
9: end for
10:   ▷ Pre-process edges: Resolve ties and select dominant direction
11: Let  $C \leftarrow \emptyset$    ▷ List of edge candidates
12: for  $i := 1$  To  $n$  do
13:   for  $j := i+1$  To  $n$  do
14:      $a2b \leftarrow \text{adjMatrix}[i][j]$ ,  $b2a \leftarrow \text{adjMatrix}[j][i]$ 
15:     if  $a2b = 0$  and  $b2a = 0$  then continue
16:   end if
17:   if  $a2b = b2a$  then
18:      $\text{adjMatrix}[i][j] \leftarrow 0$ ,  $\text{adjMatrix}[j][i] \leftarrow 0$    ▷ Remove tied edges
19:   else if  $a2b > b2a$  then
20:      $\text{sum} \leftarrow a2b + b2a$ ,  $\text{diff} \leftarrow a2b - b2a$ 
21:     Add  $(i \rightarrow j, \text{sum}, \text{diff})$  to  $C$ 
22:   else
23:      $\text{sum} \leftarrow a2b + b2a$ ,  $\text{diff} \leftarrow b2a - a2b$ 
24:     Add  $(j \rightarrow i, \text{sum}, \text{diff})$  to  $C$ 
25:   end if
26: end for
27: end for   ▷ Build final DAG using adapted Kruskal
28: Sort  $C$  by descending  $(\text{sum}, \text{diff})$ , ascending  $(u, v)$ 
29: Initialize  $\text{DAG}[n][n]$  and  $\text{Reach}[n][n] := \text{false}$ 
30: for each edge  $(u \rightarrow v, \text{sum}, \text{diff})$  in  $C$  do
31:   if  $\text{Reach}[v][u] = \text{true}$  then   ▷ Skip if creates cycle
32:     continue
33:   end if
34:   Add  $(u \rightarrow v)$  to  $\text{DAG}$ 
35:   Update transitive closure:
36:    $\text{Reach}[u][v] \leftarrow \text{true}$ 
37:   for  $i := 1$  To  $n$  do
38:     if  $\text{Reach}[i][u]$  then  $\text{Reach}[i][v] \leftarrow \text{true}$ 
39:   end if
40:     if  $\text{Reach}[v][i]$  then  $\text{Reach}[u][i] \leftarrow \text{true}$ 
41:   end if
42:     for  $j := 1$  To  $n$  do
43:       if  $\text{Reach}[i][u] \wedge \text{Reach}[v][j]$  then  $\text{Reach}[i][j] \leftarrow \text{true}$ 
44:     end if
45:   end for
46:   end for
47: end for
48: return  $\text{DAG}$ 

```

Our implementation of step 2 uses a single adjacency matrix, in which the occurrences of each edge in all local structures are recorded. For example, if in three of the local structures there is an edge ij , then the position $[i, j]$ of the adjacency matrix will contain the number 3. Likewise, if the value 3 is in position $[j, i]$ of the matrix, this indicates that all edges between these two nodes

cancel each other out and will not be present in the final DAG. However, if the value at position $[i, j]$ of the matrix is greater than the value at $[j, i]$, this means that the final DAG will contain the edge ij . The reverse is also true: If $[j, i]$ has a value greater than $[i, j]$, then the final DAG will contain the edge ji , as long as ij or ji does not generate a cycle.

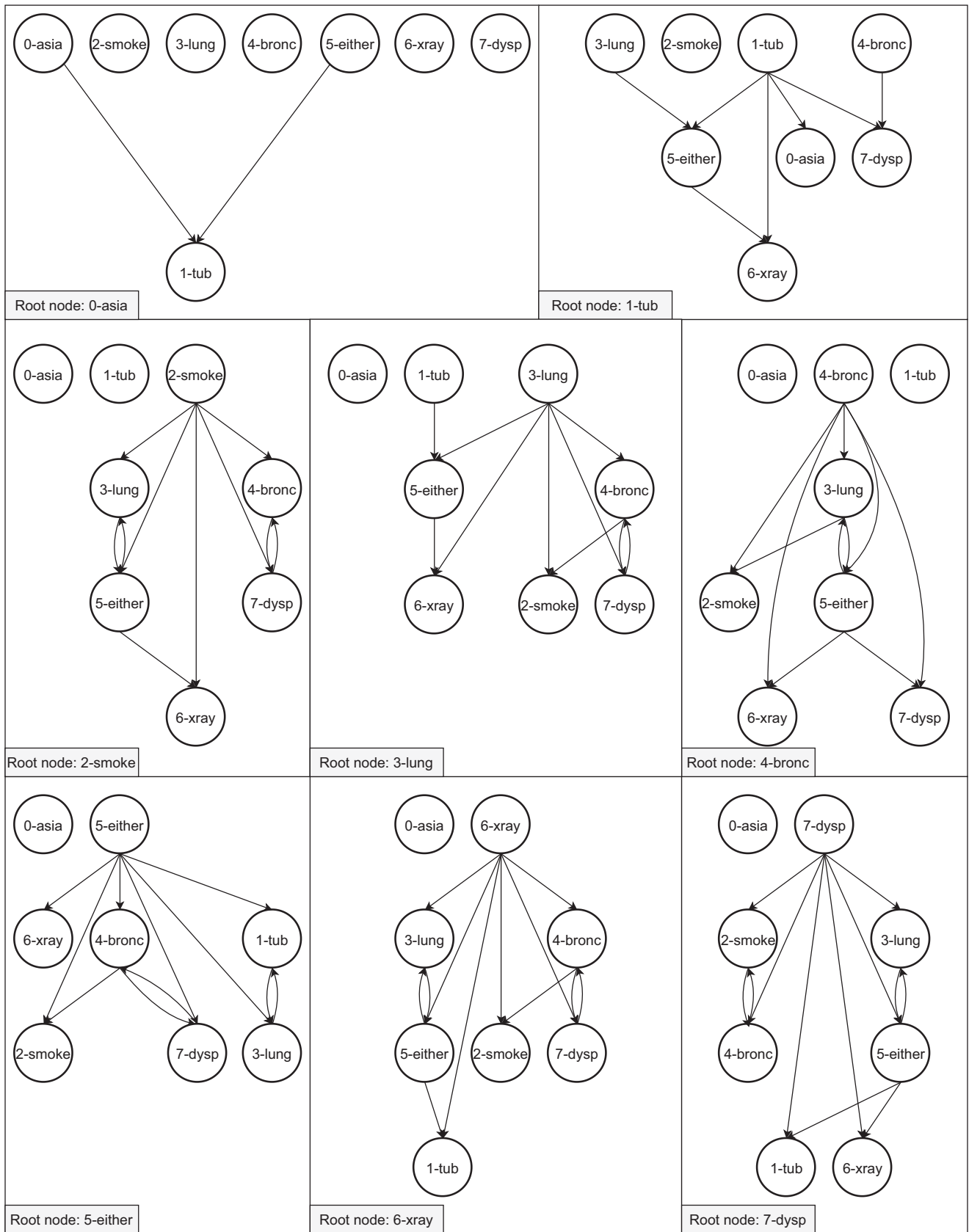


FIGURE 2 | Local network structures generated by DMBBN, with the help of the *Modified-DMBC* algorithm, from each variable of the asia dataset and its Markov blanket.

ALGORITHM 4 | Modified-DMBC algorithm.

Require: Set of n nodes, a node ordering (the first node is the root node), an upper bound u on the number of parents a node may have, and a dataset D containing m cases.

Ensure: For each node, a printout of the parents of the node.

```

1:  $CMB := \{\}$   $\triangleright$  (Markov blanket for the first node)
2: for  $i := 1$  To  $n$  do
3:    $\pi_i := \{\}$ 
4:    $P_{old} := g(i, \pi_i)$  (see Eq. 1)
5:    $OKToProceed := \text{true}$ 
6:   while  $OKToProceed$  and  $\pi_i < u$  do
7:  $\triangleright$  look for parents for node  $x_i$  in all node ordering
   and not just to the left of  $x_i$ 
8:   let  $z$  be the node in all node ordering that
   maximizes  $g(i, \pi_i \cup \{z\})$ 
9:    $P_{new} = g(i, \pi_i \cup \{z\})$ 
10:  if  $P_{new} > P_{old}$  then
11:    if ( $z$  is parent of any class child) or
    ( $z$  is a class child) then
12:       $CMB := CMB \cup \{z\}$ 
13:       $P_{old} := P_{new}$ 
14:       $\pi_i := \pi_i \cup \{z\}$ 
15:    end if
16:    if (not_class_child( $i$ ) and class  $\notin \pi_i$ ) then
17:      break
18:    end if
19:  else
20:     $OKToProceed := \text{false}$ 
21:  end if
22: end while
23: write('Node:',  $x_i$ , 'Parents of this node:',  $\pi_i$ )
24: end for

```

Finally, the final DAG is constructed using an adapted Kruskal algorithm that considers edge frequencies as weights, adds edges in descending order of importance, and maintains acyclicity through transitive closure verification.

This approach ensures that the resulting final DAG captures meaningful relationships between variables while eliminating redundant or contradictory edges that may arise from combining local structures. This step is crucial for obtaining a compact and coherent representation of the final DAG of the Bayesian network.

Figure 3 displays the complete structure of the asia network induced by DMBC from the local structures shown in Figure 2.

5 | Experiments and Analysis of Results

This section describes the experiments conducted to evaluate the performance of the proposed DMBC algorithm. The K2 algorithm was selected for comparative analysis due to its status as a widely used benchmark in structure learning. For this evaluation, we used 11 distinct datasets. These datasets were generated from Bayesian networks sourced from the *bnlearn* repository:

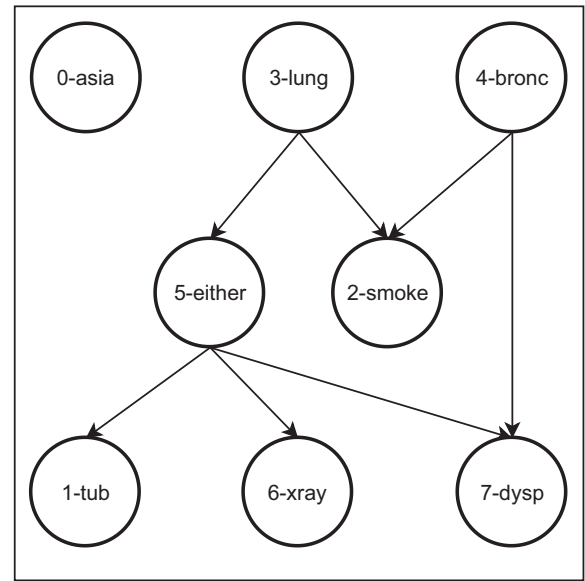


FIGURE 3 | Complete structure (DAG) of Bayesian network asia induced by DMBC from local structures.

TABLE 1 | Bayesian networks used in the experiments.

Dataset	Number of nodes	Number of edges	Number of parameters
asia	8	8	18
sachs	11	17	178
alarm	37	46	509
barley	48	84	114,005
haifinder	56	66	2656
hepar2	70	123	1453
pathfinder	109	195	72,079
andes	223	338	1157
diabetes	413	602	429,409
link	724	1125	14,211
munin	1041	1397	8059

Asia, sachs, alarm, barley, haifinder, hepar2, pathfinder, andes, diabetes, link, and munin.

The Bayesian networks selected for the experiments have different sizes, ranging from 8 to 1041 nodes and from 8 to 1397 edges. Table 1 provides more details. Datasets were artificially generated from these Bayesian networks, with ten thousand instances each, which are given as input for the execution of both DMBC and K2 algorithms.

The DMBC and K2 algorithms were implemented in the Java programming language, using the Weka library [32], and run on a computer with Intel Core i7-11390H @ 3.40GHz processor, 11th generation, memory 32GB and Linux (64-bit system).² Both algorithms were run, receiving as input each dataset and the maximum number of parents u . K2 was also given random variable orderings and was run 30 times, each with a different

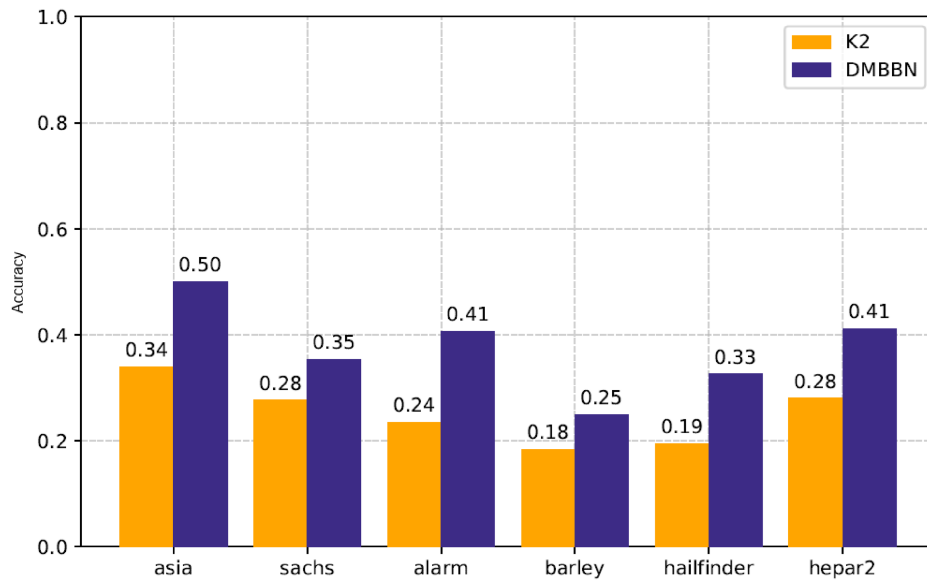


FIGURE 4 | Accuracy of the DMBBN and K2 algorithms as applied to the asia, sachs, alarm, barley, hailfinder, and hepar2 datasets.

ordering. The Bayesian network structures induced by both algorithms were then compared to the original Bayesian network structures corresponding to the datasets. For this, the following metrics were used:

- *Correct edges (C)*: Edges correctly detected by the algorithms, with the appropriate orientation, as compared with the original network structures.
- *Additional edges (A)*: Edges detected by the algorithms that are not present in the original structures.
- *Missing edges (M)*: Edges not detected by the algorithm as compared with the original structures.
- *Reverse edges (R)*: Edges detected by the algorithm, but with opposite orientation as compared with the original structures.

Clearly, the greater the number of correct edges and the smaller the number of incorrect edges, the better the algorithm.

Furthermore, the average accuracy of DMBBN was compared with the average accuracy of the K2 algorithm (Figure 4), which was calculated according to [16]:

$$\text{Accuracy} = \frac{C}{C + M + R + A}. \quad (2)$$

Tables 2 and 3 present the value of the metrics for the edges, as well as the accuracy values, obtained by the DMBBN and K2 algorithms when inducing Bayesian network structures for each of the 11 datasets used in the experiments. Values shown for K2 are averages of the 30 runs with random ordering of the variables. The best algorithm according to each metric is highlighted in bold.

A more detailed analysis of Table 2 shows that the structures induced by DMBBN are closer to the original ones when compared to the structures induced by K2 for the asia, sachs, alarm, barley, hailfinder, and hepar2 datasets. It was observed that the

number of correct edges was consistently higher in DMBBN, while the number of additional, missing, and reverse edges was lower in the vast majority of cases. Furthermore, accuracy was consistently higher in DMBBN for these six datasets, as illustrated in Figure 4, indicating more accurate learning.

Results suggest the DMBBN algorithm effectively infers the candidate parents without the need for a prior ordering of the input variables. Using well-known Bayesian networks in the literature, it was possible to compare the structure learned by DMBBN and K2 with the published structure. For those six datasets mentioned, the proposed algorithm was not only able to generate adequate structures without the need to order the attributes (which is the major limitation of K2), but also presented an average accuracy higher than the average accuracy obtained with K2 (Figure 4).

The algorithms were also run with datasets obtained from larger and more complex Bayesian networks. Five additional Bayesian networks were selected: Pathfinder, andes, diabetes, link and munin. These Bayesian networks have a considerably larger number of variables, ranging from 109 to 1041 variables. The results obtained revealed a significant drop in the accuracy of both DMBBN and K2, as evidenced in the chart in Figure 5. Perhaps the number of instances (10,000) was relatively small for those networks. Indeed, we also tried 10 DMBBN runs with the barley and hailfinder datasets, each with 10,000 different instances, and an additional 10 runs with 100,000 different instances—an increase in accuracy was observed with the larger number of instances, for both networks.

However, the columns in the chart in Figure 5 show that a single DMBBN run tends to behave similarly to thirty runs of the K2 algorithm with different random orderings. It can be seen that, in this scenario, the performance of DMBBN, in terms of accuracy, was still superior to K2 for the andes dataset (37% vs. 22%)—this experiment allowed for a greater number of parents for each variable, thus allowing the induction of a more complex structure. It

TABLE 2 | The comparison of DMBBN and K2 using the edge scores for different datasets with fewer than one hundred variables. *#Params* refers to the number of parameters; *#Parents* refers to the maximum number of parents of a node; *#Edges* refers to the number of edges; *C* (Correct); *A* (Additional); *M* (Missing); *R* (Reverse); and *Acc* (Accuracy).

Dataset	#Parents	#Edges	DMBBN	K2
asia	2	C	4	3.7
		A	0	3.23
		M	1	1.2
		R	3	3.10
		Acc	0.5	0.34
sachs	3	C	6	5.03
		A	0	1.73
		M	6	6.73
		R	5	5.23
		Acc	0.3529	0.2764
alarm	4	C	26	20.13
		A	18	40.40
		M	8	5
		R	12	20.87
		Acc	0.4063	0.2354
barley	4	C	35	22.23
		A	57	37.67
		M	35	40.60
		R	14	21.17
		Acc	0.2482	0.1840
hailfinder	4	C	43	25.40
		A	66	67.40
		M	13	19.73
		R	10	20.87
		Acc	0.3258	0.1935
hepar2	6	C	65	45.57
		A	35	41.63
		M	27	44.50
		R	31	32.93
		Acc	0.4114	0.2798

can also be seen in Table 3 that DMBBN led to better results with respect to the number of correct, reversed, and missing edges than K2 in most datasets.

These results reveal the limitations of DMBBN and K2 in learning Bayesian networks when receiving datasets with a large number of variables. Other factors, such as the number of observations and computational limits, can also impact algorithm performance. This analysis both indicates the algorithm's performance and highlights the difficulty of learning large Bayesian networks from data.

The DMBBN algorithm was also compared to four algorithms from the literature that search for a suitable ordering of nodes

TABLE 3 | The comparison of DMBBN and K2 using the edge scores for different datasets with more than one hundred variables. *#Params* refers to the number of parameters; *#Parents* refers to the maximum number of parents of a node; *#Edges* refers to the number of edges; *C* (Correct); *A* (Additional); *M* (Missing); *R* (Reverse); and *Acc* (Accuracy).

Dataset	#Parents	#Edges	DMBBN	K2
pathfinder	5	C	78	46.33
		A	667	214.13
		M	87	127.80
		R	30	20.87
		Acc	0.0905	0.1200
andes	6	C	238	149.17
		A	301	342.43
		M	34	62.40
		R	66	126.43
		Acc	0.3725	0.2196
diabetes	2	C	212	155.27
		A	1,271	376.13
		M	228	299.10
		R	162	147.63
		Acc	0.1132	0.1588
link	3	C	55	73.53
		A	7,715	673.40
		M	1,013	981.00
		R	57	70.47
		Acc	0.0062	0.0409
munin	3	C	405	315.97
		A	21,677	927.77
		M	753	852.57
		R	239	228.47
		Acc	0.0176	0.1360

for K2: The algorithms of Tabar et al. [16], Ko and Kim [28], Ai [30], and ITNO-K2 [21]. Table 4 presents the comparison of the algorithms using edge scores for asia and alarm datasets (ten thousand instances). The best algorithm according to each metric is highlighted in bold. The results of the Tabar et al., Ko and Kim, and Ai algorithms were presented in [16] and replicated in [21] along with the results of ITNO-K2. It is worth noting that the algorithms of Tabar et al., Ko and Kim, and Ai were executed 100 times, as reported in [16] and [21], while ITNO-K2 and DMBBN were run only once. According to Table 4, DMBBN can produce structures for asia as good as the other four algorithms. For alarm, DMBBN obtained better results than Ai and worse than the algorithms of Tabar et al., Ko and Kim, and ITNO-K2. However, it seems that DMBBN can be competitive in more complex scenarios, as demonstrated for the andes network. We note that Benmohamed et al. [21] state that the ITNO algorithm delivers decreased performance for larger networks.

Table 5 shows the difference in computational cost of DMBBN compared to K2, presenting the execution time (ms) and the number of calls to function *g* (which is the core of K2 and DMBBN).

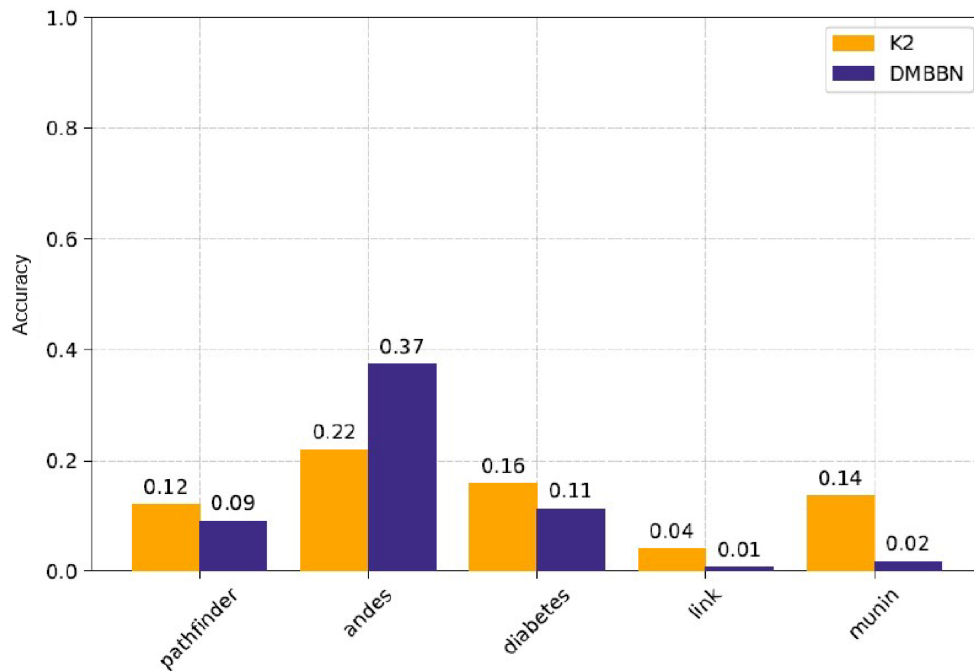


FIGURE 5 | Variation in the accuracy of the DMBBN and K2 algorithms for learning network structures from the following datasets: Pathfinder, andes, diabetes, link, and munin.

TABLE 4 | The comparison of DMBBN and other algorithms using the edge scores for asia and alarm datasets. *C* (Correct); *A* (Additional); *M* (Missing); *R* (Reverse); and *Acc* (Accuracy).

Dataset	#Edges	Tabar	Ko and Kim	Ai	ITNO-K2	DMBBN
asia	C	6	5	5	6	4
	A	1	1	3	0	0
	M	0	0	1	1	1
	R	3	3	1	1	3
	Acc	0.55	0.55	0.44	0.75	0.5
alarm	C	41	40	24	35	26
	A	7	15	30	8	18
	M	1	2	2	9	8
	R	8	4	20	2	12
	Acc	0.732	0.678	0.324	0.791	0.41

The computational cost of DMBBN was not compared to the other four algorithms from the literature because it was not possible to execute them in the same experimental setting. As expected, a single execution of DMBBN has a much higher computational effort than thirty executions of K2, as DMBBN ignores the main restriction of K2 of finding parents for a variable only to the left of it in a previous ordering. Roughly speaking, DMBBN took, on average, 169 times longer than K2 to execute the 11 datasets—4 times slower for asia and 603 times slower for munin. Taking into account the calls to the g function, DMBBN needed 77 times more calls than K2, on average: 6 times more calls to the g function than K2 for asia and 470 times more calls than K2 for munin. However, despite the higher computational cost, DMBBN can still be executed on a relatively simple computer with the configuration described above. The fact that it learns the graph without the need for a prior ordering of variables may

TABLE 5 | Time consumed during structure learning and number of calls to g function (#g) by DMBBN and K2.

Dataset	DMBBN		K2	
	Time (ms)	#g	Time (ms)	#g
asia	717	322	220	55
sachs	427	1090	66	128
alarm	7037	49,716	1452	2206
barley	42,363	69,325	2882	3244
hailfinder	49,070	106,810	2708	4972
hepar2	97,390	142,140	4533	7349
pathfinder	817,596	2,489,508	4885	20,360
andes	2,695,902	2,695,080	34,278	103,235
diabetes	9,382,545	17,408,648	45,250	163,605
link	9,370,998	13,206,318	137,577	484,994
munin	184,582,658	538,938,400	336,085	1,146,365

be a compensation for the computational cost required in complex environments. The choice between the algorithms must take into account not only the processing time, but also the specific requirements of the application in question.

6 | Conclusions

A key challenge in learning Bayesian networks in a real setting is the difficulty of defining a variable ordering that accurately reflects the underlying causal structure, especially when prior knowledge is scarce. We present a novel solution, the Dynamic Markov Blanket Bayesian Network (DMBBN) algorithm, which

learns the network structure from data independently of any prior variable ordering. DMBBN induces the final Directed Acyclic Graph (DAG) by joining local structures, each derived efficiently from the Markov blanket of its respective domain variable.

The Dynamic Markov Blanket Bayesian Network (DMBBN) algorithm was evaluated empirically across eleven datasets containing between 8 and 1041 variables. Comparative experiments were performed against the established K2 algorithm. The main conclusions drawn from this work are: (i) DMBBN consistently demonstrated better efficiency than K2 in inducing network structures across the majority of the tested datasets; (ii) Even in datasets with more than one hundred variables, a single run of DMBBN produced results similar to the average of thirty runs of K2 receiving random orderings. Given these experimental results, DMBBN seems a viable alternative to K2, especially for datasets characterized by a large number of variables (over one hundred), as it completely eliminates the need to seek a suitable variable ordering.

Unlike algorithms such as those proposed by Tabar et al., Ko and Kim, Ai, and ITNO-K2, which rely on optimizing a variable ordering, DMBBN is promising as it directly induces the Directed Acyclic Graph (DAG) without requiring a predefined variable ordering.

In future work, we intend to investigate the ability of DMBBN to handle cycles detected in data, so as to induce cyclic structural causal models.

Acknowledgments

The authors would like to thank the Federal University of São João del-Rei (UFSJ) for supporting this project through Notice 006/2023/PROPE-PIBIC/Fapemig/UFSJ. Fabio Gagliardi Cozman thanks CNPq (Grant No. 305753/2022-3) and the Center for AI at Universidade de São Paulo, funded by FAPESP (Grant No. 2019/07665-4) and IBM.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in Livia-Carvalho at: <https://github.com/Livia-Carvalho>

Endnotes

¹ Available at: <https://www.bnlearn.com/bnrepository/>.

² The codes and datasets are available at: <https://github.com/Livia-Carvalho>.

References

1. N. K. Kitson, A. C. Constantinou, Z. Guo, Y. Liu, and K. Chobtham, "A Survey of Bayesian Network Structure Learning," *Artificial Intelligence Review* 56, no. 8 (2023): 8721–8814.
2. J. Pearl and D. Mackenzie, *The Book of Why: The New Science of Cause and Effect*, 1st ed. (Basic Books, Inc., 2018).

3. J. Pearl, "Bayesian Networks: A Model of Self-Activated Memory for Evidential Reasoning," in *Proceedings of the 7th Conference of the Cognitive Science Society*, Irvine, UCLA Technical Report CSD-850017 (1985), 15–17, http://ftp.cs.ucla.edu/tech-report/198_-reports/850017.pdf.
4. J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference* (Morgan Kaufmann Publishers Inc., 1988).
5. K. Sachs, O. Perez, D. Pe'er, D. A. Lauffenburger, and G. P. Nolan, "Causal Protein-Signaling Networks Derived From Multiparameter Single-Cell Data," *Science* 308, no. 5721 (2005): 523–529, <https://doi.org/10.1126/science.1105809>.
6. S. Imoto, T. Higuchi, T. Goto, K. Tashiro, S. Kuhara, and S. Miyano, "Combining Microarrays and Biological Knowledge for Estimating Gene Networks via Bayesian Networks," *Journal of Bioinformatics and Computational Biology* 2, no. 1 (2004): 77–98, <https://doi.org/10.1142/S021972000400048X>.
7. G. Moffa, G. Catone, J. Kuipers, et al., "Using Directed Acyclic Graphs in Epidemiological Research in Psychosis: An Analysis of the Role of Bullying in Psychosis," *Schizophrenia Bulletin* 43, no. 6 (2017): 1273–1279, <https://doi.org/10.1093/schbul/sbx013>.
8. M. B. Sesen, A. E. Nicholson, R. Banares-Alcantara, T. Kadir, and M. Brady, "Bayesian Networks for Clinical Decision Support in Lung Cancer Care," *PLoS One* 8, no. 12 (2013), <https://doi.org/10.1371/journal.pone.0082349>.
9. B. Cai, L. Huang, and M. Xie, "Bayesian Networks in Fault Diagnosis," *IEEE Transactions on Industrial Informatics* 13, no. 5 (2017): 2227–2240, <https://doi.org/10.1109/TII.2017.2695583>.
10. C. Vitolo, M. Scutari, M. Ghalaieny, A. Tucker, and A. Russell, "Modeling Air Pollution, Climate, and Health Data Using Bayesian Networks: A Case Study of the English Regions," *Earth and Space Science* 5, no. 4 (2018): 76–88, <https://doi.org/10.1002/2017EA000326>.
11. E. Bareinboim, J. D. Correa, D. Ibeling, and T. Icard, "On Pearl's Hierarchy and the Foundations of Causal Inference," in *Probabilistic and Causal Inference: The Works of Judea Pearl*, 1st ed. (Association for Computing Machinery, 2022), 507–556.
12. C. C. Emezue, A. Drouin, T. Deleu, S. Bauer, and Y. Bengio, "Benchmarking Bayesian Causal Discovery Methods for Downstream Treatment Effect Estimation," in *ICML Workshop on Structured Probabilistic Inference and Generative Modeling* (2023).
13. D. Chickering, D. Geiger, and D. Heckerman, *Learning Bayesian Networks is NP-Hard. Technical Report MSR-TR-94-17*, vol. 196 (Microsoft Research, 1994).
14. M. Chickering, D. Heckerman, and C. Meek, "Large-Sample Learning of Bayesian Networks Is NP-Hard," *Journal of Machine Learning Research* 5 (2004): 1287–1330.
15. R. E. Neapolitan, *Learning Bayesian Networks*, vol. 38 (Pearson Prentice Hall Upper Saddle River, NJ, 2004).
16. V. R. Tabar, F. Eskandari, S. Salimi, and H. Zareifard, "Finding a Set of Candidate Parents Using Dependency Criterion for the K2 Algorithm," *Pattern Recognition Letters* 111 (2018): 23–29.
17. P. Spirtes and C. Glymour, "An Algorithm for Fast Recovery of Sparse Causal Graphs," *Social Science Computer Review* 9, no. 1 (1991): 62–72.
18. G. F. Cooper and E. Herskovits, "A Bayesian Method for the Induction of Probabilistic Networks From Data," *Machine Learning* 9, no. 4 (1992): 309–347.
19. E. B. dos Santos, E. R. Hruschka, Jr., E. R. Hruschka, and N. F. F. Ebecken, "Bayesian Network Classifiers: Beyond Classification Accuracy," *Intelligent Data Analysis* 15, no. 3 (2011): 279–298, <https://doi.org/10.3233/IDA-2010-0468>.
20. M. Scutari, C. E. Graafland, and J. M. Gutiérrez, "Who Learns Better Bayesian Network Structures: Accuracy and Speed of Structure Learning

Algorithms,” *International Journal of Approximate Reasoning* 115 (2019): 235–253, <https://doi.org/10.1016/j.ijar.2019.10.003>.

21. E. Benmohamed, H. Ltfi, and M. B. Ayed, “ITNO-K2PC: An Improved K2 Algorithm With Information-Theory-Centered Node Ordering for Structure Learning,” *Journal of King Saud University, Computer and Information Sciences* 34, no. 4 (2022): 1410–1422.

22. N. K. Kitson and A. C. Constantinou, “The Impact of Variable Ordering on Bayesian Network Structure Learning,” 2024.

23. P. Larranaga, C. M. Kuijpers, R. H. Murga, and Y. Yurramendi, “Learning Bayesian Network Structures by Searching for the Best Ordering With Genetic Algorithms,” *IEEE Transactions on Systems, Man, and Cybernetics* 26, no. 4 (1996): 487–493.

24. W. H. Hsu, H. Guo, B. B. Perry, and J. A. Stilson, “A Permutation Genetic Algorithm for Variable Ordering in Learning Bayesian Networks From Data,” in *Proceedings of the 4th Annual Conference on Genetic and Evolutionary Computation* (Morgan Kaufmann Publishers Inc., 2002), 383–390.

25. T. Romero, P. Larrañaga, and B. Sierra, “Learning Bayesian Networks in the Space of Orderings With Estimation of Distribution Algorithms,” *International Journal of Pattern Recognition and Artificial Intelligence* 18, no. 4 (2004): 607–625, <https://doi.org/10.1142/S0218001404003332>.

26. J. E. R. Hruschka and N. F. Ebecken, “Towards Efficient Variables Ordering for Bayesian Networks Classifier,” *Data & Knowledge Engineering* 63, no. 2 (2007): 258–269.

27. X. W. Chen, G. Anantha, and X. Lin, “Improving Bayesian Network Structure Learning With Mutual Information-Based Node Ordering in the K2 Algorithm,” *IEEE Transactions on Knowledge and Data Engineering* 20, no. 5 (2008): 628–640.

28. S. Ko and D. W. Kim, “An Efficient Node Ordering Method Using the Conditional Frequency for the K2 Algorithm,” *Pattern Recognition Letters* 40 (2014): 80–87, <https://doi.org/10.1016/j.patrec.2013.12.021>.

29. R. Aghdam, M. Ganjali, X. Zhang, and C. Eslahchi, “CN: A Consensus Algorithm for Inferring Gene Regulatory Networks Using the SORDER Algorithm and Conditional Mutual Information Test,” *Molecular BioSystems* 11 (2015): 942–949, <https://doi.org/10.1039/C4MB00413B>.

30. X. Ai, “Node Importance Ranking of Complex Networks With Entropy Variation,” *Entropy* 19, no. 7 (2017): 303, <https://doi.org/10.3390/e19070303>.

31. S. Behjati and H. Beigy, “Improved K2 Algorithm for Bayesian Network Structure Learning,” *Engineering Applications of Artificial Intelligence* 91 (2020): 103617.

32. I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining, Fourth Edition: Practical Machine Learning Tools and Techniques*, 4th ed. (Morgan Kaufmann Publishers Inc, 2016).

Supporting Information

Additional supporting information can be found online in the Supporting Information section. **Data S1.** Supporting Information.